Check for updates

TRANSACTIONS OF THE INSTITUTE OF MEASUREMENT & CONTROL

Article - Control

Transactions of the Institute of Measurement and Control
1–18
© The Author(s) 2025

Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/01423312251366726
journals.sagepub.com/home/tim
Sage

Intelligent robust control design for a 3-DoF industrial manipulator

Loc Nguyen[1], Khoi Pham[1] and Thanh Tran[2]

Abstract

Precise control of multi-degree-of-freedom (DoF) robotic manipulators is essential in industrial settings, where high tracking accuracy, fast convergence, and robustness to uncertainty are critical. However, many existing methods continue to suffer from high chattering, slow response, and limited adaptability to disturbances or abrupt input variations. This paper presents a control framework for a 3-DoF industrial manipulator that integrates nonsingular terminal sliding mode control (NTSMC), backstepping design, and a clustering-enhanced radial basis function neural network (RBFNN). The NTSMC component provides finite-time convergence and strong robustness to matched uncertainties, leading to superior tracking performance with a root mean squared error (RMSE) of 0.000202 under ideal conditions and 0.002634 under disturbances, both significantly better than those achieved by Fuzzy-sliding mode control (SMC) and SMC-neural network (NN) baselines. Backstepping enables smooth coordination of the control input across dynamic subsystems, which reduces settling time to 0.0375 seconds in the ideal case and 0.2808 seconds under disturbance. The clustering-enhanced RBFNN allows structured and efficient initialization of network parameters, identifying effective centers and widths in fewer than 500 iterations and improving generalization to both periodic and abrupt input signals. The combined design eliminates the common trade-off between chattering suppression and trajectory smoothness and demonstrates practical potential for real-time, high-precision control in uncertain industrial environments.

Keywords

3-DoF manipulator, adaptive control, backstepping SMC, chattering reduction, Lyapunov stability, neural network function approximation, nonsingular terminal SMC, RBF neural network

Introduction

Robust control strategies have been extensively applied to industrial robotic manipulators due to their ability to ensure system stability under uncertainties, model inaccuracies, and external disturbances. Among these methods, sliding mode control (SMC) has gained prominence by enforcing the system state to reach and remain on a designed sliding manifold, thereby ensuring robustness to matched uncertainties (Su, 2025; Xu et al., 2021). However, conventional SMC suffers from chattering, a high-frequency oscillation in the control signal caused by its discontinuous nature (Slotine and Li, 1991). This not only deteriorates control precision but also imposes mechanical stress on actuators. In addition, classical SMC does not guarantee finite-time convergence, which limits its effectiveness in applications requiring fast and accurate tracking (Khalil, 2002).

To improve convergence speed and reduce chattering, nonsingular terminal sliding mode control (NTSMC) introduces nonlinear sliding surfaces incorporating both the tracking error and its rate of change. This design enhances the adaptability of the switching law and enables finite-time convergence (Su, 2025; Zhu et al., 2023). Nevertheless, most existing NTSMC frameworks are limited to lower-order systems with relatively simple dynamics and do not scale well to highly nonlinear, coupled multi-degree-of-freedom (DoF) manipulators (Xu et al., 2021).

To address this limitation, backstepping control has been developed as a recursive design methodology for stabilizing high-order nonlinear systems. By decomposing the original system into a sequence of lower-order subsystems and constructing virtual control laws, backstepping ensures the Lyapunov stability at each design step and ultimately guarantees global asymptotic stability (Vaidyanathan and Azar, 2020). This makes it especially suitable for complex robotic manipulators with significant dynamic coupling. However, a key drawback of Backstepping is its dependence on accurate system models, which may not be available or reliable in many practical scenarios (Capone and Hirche, 2019; Ibarra et al., 2023).

In response to model uncertainty, radial basis function neural networks (RBFNNs) are widely employed for real-time approximation of unknown system dynamics and disturbances (Liu, 2013, 2017). RBFNNs offer universal function approximation capabilities without requiring prior structural knowledge (Hagan et al., 2014). However, standard RBFNNs often require a large number of neurons and suffer from the lack of a systematic method for selecting optimal parameters, which

[1]Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
[2]School of Science, Engineering & Technology, RMIT University Vietnam, Vietnam

Corresponding author:
Thanh Tran, School of Science, Engineering & Technology, RMIT University Vietnam, 702 Nguyen Van Linh Boulevard, Tan Hung Ward, Ho Chi Minh City 700000, Vietnam.
Email: thanh.tran37@rmit.edu.vn

Data Availability Statement included at the end of the article

can lead to overfitting and increase computational complexity (Vachkov et al., 2014).

To alleviate this, data-driven clustering techniques can be used to configure the RBF network structure by grouping similar input samples, thereby reducing redundancy and ensuring a compact yet effective network. This approach significantly improves learning efficiency and allows for practical deployment in real-time control applications (Liu, 2013).

Motivated by the limitations of conventional robust control methods and the need for precise, adaptive control in industrial robotic manipulators, this paper proposes a novel control framework that integrates:

- Backstepping control, to manage the complexity of multi-DoF robotic systems via recursive design.
- NTSMC, to achieve robustness and finite-time convergence while mitigating chattering.
- RBFNNs, to adaptively approximate unknown nonlinearities and external disturbances without requiring prior system knowledge.
- A clustering-based RBF optimization scheme, to improve parameter initialization and enhance adaptation efficiency.

The proposed hybrid control strategy is applied to a three-degree-of-freedom (3-DoF) robotic manipulator subject to dynamic coupling, nonlinearities, and external disturbances. Extensive simulation results are presented to validate the effectiveness of the method, demonstrating superior trajectory tracking accuracy, robustness, and adaptability compared to conventional approaches.

This study begins with a review of primary contributions in the fields of robotics and control. The problem formulation and the complete model of the robotic arm are subsequently presented. Following this, the control law design is introduced, with a focus on enhancing stability and robustness. The theoretical background of RBFNNs is then discussed, along with strategies for their integration. Next, the adaptive law is developed, accompanied by an analysis of system stability and convergence. The simulation setup is described in detail, including all conditions and parameter values. The results are then analyzed and compared against theoretical expectations. Finally, the study concludes with a summary of key findings and directions for future research.

Literature review

This section reviews key studies on advanced SMC and adaptive control, focusing on NTSMC and its variants that integrate backstepping and RBFNN models. The review highlights their effectiveness and practical relevance to industrial robotics.

Jin et al. (2024) proposed a Fast NTSMC combined with an RBFNN model for insulator cleaning robots. The method focuses on tracking the sliding surface speed $\dot{e}$, ensures finite-time convergence, and reduces the high chattering found in conventional switching laws. It improves convergence time by 20% in both ideal and disturbed conditions compared to traditional and adaptive NTSMC. However, the system still exhibits relatively high tracking error and is constrained by a limited model structure.

Li et al. (2024) extended the work of Jin et al. (2024) by proposing a Practical Predefined-Time Fast NTSMC for a five-degree-of-freedom (5-DoF) upper-limb exoskeleton designed to assist human motor function. To improve tracking accuracy and ensure patient safety, the method imposes constraints on time and speed, enabling manual control of the manipulator as needed. It reduced the positional tracking error to below 10^{-5} and ensured stable initial conditions for safe operation. However, a trade-off between control input amplitude and convergence speed remains, indicating the need for further study under input saturation.

Duan et al. (2023) improved the aforementioned works by introducing the adaptive leader salp swarm algorithm (ALSSA), a parameter optimization method for NTSMC with RBFNN applied to an ABB robotic arm. The approach uses swarm behavior based on leader–follower dynamics to optimize parameter tuning. Experimental results demonstrated a 97% reduction in maximum steady-state error and a high success rate in pick-and-place tasks. Despite these gains, the system may generate large initial control torques, which could pose a risk to the physical hardware.

Xia et al. (2022) applied NTSMC-RBFNN control with Control Moment Gyros (CMGs) for real-time adjustment of a space robot. CMGs function as reactionless actuators to reduce dynamic coupling between platforms. The method is designed specifically for space applications, incorporating a kinematic controller for both position and attitude regulation. Experimental results showed precise signal tracking within 0.01° and a reduction in tracking error to 0.5×10^{-3}. Although performance improved significantly, residual disturbances persisted, and the added complexity from CMGs remains a concern.

Yuan et al. (2024) developed a hybrid control method that combines proportional–integral–derivative (PID), feedforward, and RBFNN-based integral sliding mode control (ISMC) for motion tracking in a lower-limb exoskeleton robot. To ensure safe patient movement, the sliding surface was extended to include acceleration, velocity, and position, forming an adaptive ISMC capable of eliminating residual error and suppressing high-frequency oscillations. Simulation results showed that the method outperformed traditional PID-feedforward (PIDFF) control, achieving a root mean square (RMS) tracking error of 8.79041×10^{-4}, nearly 10 times lower than the 0.0083 obtained with PIDFF. However, the system remains highly sensitive to parameter tuning.

The reviewed studies demonstrate the strong potential of NTSMC combined with RBFNN in various robotic applications. Building on this foundation, our proposed method is expected to offer a promising solution to related control tasks.

Contributions

The literature review highlights control strategies that improve precision, robustness, and adaptability in robotic systems.

Table 1. Comparison of the proposed method with related work.

Authors	Techniques	Limitations	Our advantages
Jin et al. (2024)	Fast NTSMC-RBFNN	Challenges remain due to relatively high error and limitations of the simplified model	Significantly reduce convergence time to 0.037533 seconds compared to 1.83 seconds; overcomes high error issues by integrating a clustering-enhanced RBFNN that adapts to complex dynamics
Li et al. (2024)	Practical Predefined-Time Fast NTSMC	Trade-off between control input amplitude and convergence speed	Addresses input saturation by incorporating a constraint-aware control design that balances convergence speed and control effort
Duan et al. (2023)	Fast NTSMC-RBFNN; ALSSA	Subject to large initial control torques, which may risk damage to the physical system	Reduces large initial torques through a smooth-start backstepping design
Xia et al. (2022)	NTSMC-RBFNN with CMGs	Residual disturbances persist despite CMG-based reduction, adding to overall system complexity	Mitigates residual disturbances using a clustering-enhanced RBFNN structure that improves disturbance rejection; significantly reduces RMSE tracking error to below 10^{-4}, compared to 0.5×10^{-3}
Yuan et al. (2024)	PID-ISMC-RBFNN	Relies on highly accurate parameter tuning; initial steady-state errors remain present	Alleviates reliance on precise parameter tuning through self-organizing RBFNN clustering that auto-adjusts to task dynamics

Nonetheless, optimal performance under dynamic and uncertain conditions remains a challenge. This study addresses these gaps with the following contributions:

- We develop a dynamic model for a 3-DoF articulated robot arm based on the first three joints of the IRB1600 industrial robot (ABB Robotics, 2024). While most studies focus on two-degree-of-freedom (2-DoF) systems, this work extends control strategies to higher-DoF models, narrowing the gap between theoretical research and industrial practice.
- We propose a control system that combines NTSMC, backstepping, and an RBFNN-based adaptive law to improve adaptation, enhance tracking accuracy, and reduce chattering. Stability is established using the Lyapunov theory, and simulations confirm robust performance and superior tracking compared to other adaptive methods. A brief comparison highlighting the advantages of the proposed method over the approaches reviewed in the literature is presented in Table 1.
- To improve RBFNN performance, we introduce a clustering algorithm that optimally initializes the network's centers and widths. By capturing both spatial and temporal characteristics of the input data, the method enables accurate modeling of the system dynamics as well as disturbances.

Problem statement

We constructed a dynamic model to replicate the behavior of a 3-DoF articulated robotic arm, utilizing the first three joints of the IRB1600 (ABB Robotics, 2024) industrial robot. While the Newton–Euler method can be computational advantages for real-time control in systems with limited degrees of freedom (Furlani, 2022), the Lagrange–Euler method is much more advantageous for complex multi-DoF system with constraints as they utilize generalized coordination and simplify the derivation of equations of motion (Lynch and Park, 2017; Murray et al., 1994). Although constraints are not explicitly discussed in this paper, the main focus would be that control can help robot arm in industrial setting where multiple DoF system navigating in constrained environments. The resulting system dynamics are then expressed as a second-order model using the Lagrange–Euler method, as follows:

$$\mathbf{M}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{g}(\mathbf{q}) + \mathbf{f}(\dot{\mathbf{q}}) = \boldsymbol{\tau}(t) + \mathbf{p}(t), \tag{1}$$

where

- $\mathbf{q} \in \mathbb{R}^{3\times 1}$, $\dot{\mathbf{q}} \in \mathbb{R}^{3\times 1}$, $\ddot{\mathbf{q}} \in \mathbb{R}^{3\times 1}$ represent the joint position, velocity, and acceleration vectors, respectively;
- $\mathbf{M}(\mathbf{q}) = \mathbf{M}_0(\mathbf{q}) + \Delta\mathbf{M}(\mathbf{q}) \in \mathbb{R}^{3\times 3}$ denotes the mass matrix of the robot's manipulator;
- $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \mathbf{C}_0(\mathbf{q}, \dot{\mathbf{q}}) + \Delta\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \in \mathbb{R}^{3\times 3}$ presents the Coriolis and centrifugal torque matrix;
- $\mathbf{g}(\mathbf{q}) = \mathbf{g}_0(\mathbf{q}) + \Delta\mathbf{g}(\mathbf{q}) \in \mathbb{R}^{3\times 1}$ refers to the gravitational torque vector;
- $\mathbf{f}(\dot{\mathbf{q}}) = \mathbf{f}_0(\dot{\mathbf{q}}) + \Delta\mathbf{f}(\dot{\mathbf{q}}) \in \mathbb{R}^{3\times 1}$ describes the frictional torque acting on the joints;
- $\boldsymbol{\tau}(t) \in \mathbb{R}^{3\times 1}$ indicates the control torque vector; and
- $\mathbf{p}(t) \in \mathbb{R}^{3\times 1}$ models the bounded internal and external disturbances, as well as parameter uncertainties.

Details of the components of the specified matrices and vectors are provided in Appendix A. In addition, the disturbance

term can be broken down as follows:

$$\mathbf{p}(t) = \mathbf{d}(t) - \Delta\mathbf{M}(\mathbf{q})\,\ddot{\mathbf{q}} - \Delta\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})\,\dot{\mathbf{q}} - \Delta\mathbf{g}(\mathbf{q}) - \Delta\mathbf{f}(\dot{\mathbf{q}}), \tag{2}$$

where

- $\mathbf{d}(t) \in \mathbb{R}^{3\times 1}$ denotes the external disturbance vector resulting from environmental factors; and
- $\Delta\mathbf{M}(\mathbf{q})$, $\Delta\mathbf{C}(\mathbf{q},\dot{\mathbf{q}})$, $\Delta\mathbf{g}(\mathbf{q})$, and $\Delta\mathbf{f}(\dot{\mathbf{q}})$ represent the internal model uncertainties.

Control law design

Backstepping NTSMC design

We begin by defining $\mathbf{x}_1 = \mathbf{q}$, $\mathbf{x}_2 = \mathbf{q}$. Using these definitions, the dynamic equation given in equation (1) can be rewritten in a strict feedback form as follows:

$$\mathbf{x}_1 = \mathbf{q}, \tag{3}$$

$$\dot{\mathbf{x}}_1 = \mathbf{x}_2 = \dot{\mathbf{q}}, \tag{4}$$

$$\ddot{\mathbf{x}}_1 = \dot{\mathbf{x}}_2 = \ddot{\mathbf{q}}, \tag{5}$$

$$\dot{\mathbf{x}}_2 = \mathbf{M}^{-1}\left[\boldsymbol{\tau} + \mathbf{p} - \mathbf{C}\mathbf{x}_2 - \mathbf{g} - \mathbf{f}\right], \tag{6}$$

and

$$\mathbf{y} = \mathbf{x}_1 = \mathbf{q}. \tag{7}$$

The control input and output are defined as $\boldsymbol{\tau} = [\tau_1, \tau_2, \tau_3]^T$ and $\mathbf{y} = \mathbf{q} = [q_1, q_2, q_3]^T$, respectively. Let $\mathbf{e}_1$ denote the position tracking error:

$$\mathbf{e}_1 = \mathbf{x}_1 - \mathbf{q}_d, \tag{8}$$

where $\mathbf{q}_d = [q_{1d}, q_{2d}, q_{3d}]^T$ presents the reference signal vector. We then have:

$$\dot{\mathbf{e}}_1 = \mathbf{x}_2 - \dot{\mathbf{q}}_d. \tag{9}$$

To stabilize the state $\mathbf{x}_1$, we introduce a virtual control $\boldsymbol{\alpha}_1$ and set $\mathbf{x}_2 = \boldsymbol{\alpha}_1$. The Lyapunov function for the first state is chosen as:

$$V_1 = \frac{1}{2}\mathbf{e}_1^T\mathbf{e}_1. \tag{10}$$

Inserting equation (9) into the time derivative of V_1, we obtain:

$$\dot{V}_1 = \mathbf{e}_1^T(\boldsymbol{\alpha}_1 - \dot{\mathbf{q}}_d). \tag{11}$$

To ensure $\dot{V}_1 < 0$ for all $\mathbf{e}_1 \neq 0$, we select the virtual controller $\boldsymbol{\alpha}_1$ as:

$$\boldsymbol{\alpha}_1 = \dot{\mathbf{q}}_d - c_1\mathbf{e}_1. \tag{12}$$

where c_1 is a positive constant. Substituting this into equation (11), we obtain:

$$\dot{V}_1 = -c_1\mathbf{e}_1^T\mathbf{e}_1. \tag{13}$$

This implies $\dot{V}_1 \leq 0$. As a result, the first control law is asymptotically stable.

In the next stage, a controller $\boldsymbol{\tau}(t)$ is selected to stabilize the state $\mathbf{x}_2$. First, we define the error $\mathbf{e}_2$ as:

$$\mathbf{e}_2 = \mathbf{x}_2 - \boldsymbol{\alpha}_1. \tag{14}$$

Taking the time derivative of the tracking error $\mathbf{e}_2$ yields:

$$\dot{\mathbf{e}}_2 = \mathbf{M}^{-1}\left[\boldsymbol{\tau} + \mathbf{p} - \mathbf{C}\mathbf{x}_2 - \mathbf{g} - \mathbf{f}\right] - \dot{\boldsymbol{\alpha}}_1. \tag{15}$$

The standard SMC controller involves differentiating the sliding manifold $\mathbf{s}$, which may result in negative powers, causing unbounded control signals and practical challenges. To avoid these problems, we opt for a nonsingular terminal sliding surface, defined as follows:

$$\mathbf{s} = \mathbf{e}_1 + \lambda\mathbf{e}_2^{\gamma/\sigma}, \tag{16}$$

where $1 < \gamma/\sigma < 2$ and γ, σ, and λ are the positive numbers. The time derivative of $\mathbf{s}$ is given by:

$$\dot{\mathbf{s}} = \dot{\mathbf{e}}_1 + \boldsymbol{\beta}\dot{\mathbf{e}}_2, \tag{17}$$

where $\boldsymbol{\beta} = \lambda\frac{\gamma}{\sigma}\mathrm{diag}\left(\mathbf{e}_2^{\gamma/\sigma-1}\right)$. The equivalent control $\boldsymbol{\tau}_{\mathrm{eq}}$ can be found by setting $\dot{\mathbf{s}} = 0$:

$$\boldsymbol{\tau}_{\mathrm{eq}} = -\mathbf{M}\boldsymbol{\beta}^{-1}\dot{\mathbf{e}}_1 - \widehat{\mathbf{p}} + \left[\mathbf{C}\mathbf{x}_2 + \mathbf{g} + \mathbf{f}\right] + \mathbf{M}\dot{\boldsymbol{\alpha}}_1. \tag{18}$$

here, $\widehat{\mathbf{p}}$ is an approximation of $\mathbf{p}$. The distance between them is assumed to be norm-bounded by a known upper limit:

$$||\mathbf{p} - \widehat{\mathbf{p}}||_i \leq \bar{p}_i, \tag{19}$$

where $i = 1, \cdots, \infty$. We choose the following control law:

$$\boldsymbol{\tau} = \boldsymbol{\tau}_{\mathrm{eq}} + \boldsymbol{\tau}_{\mathrm{sw}}, \tag{20}$$

where

- The equivalent term $\boldsymbol{\tau}_{\mathrm{eq}}$ has been deduced in equation (18);
- The term $\boldsymbol{\tau}_{\mathrm{sw}}$ denotes the switching control and is initially chosen as:

$$\boldsymbol{\tau}_{\mathrm{sw}} = -K\mathrm{sign}(\mathbf{s}), \tag{21}$$

where K serves as a positive constant.

Define the Lyapunov function for the second stage as:

$$V_2 = V_1 + \frac{1}{2}\mathbf{s}^T\mathbf{M}\boldsymbol{\beta}^{-1}\mathbf{s}. \tag{22}$$

Taking the time derivative of V_2 yields:

$$\dot{V}_2 = -c_1 \mathbf{e}_1^T \mathbf{e}_1 + \mathbf{s}^T \mathbf{M} \boldsymbol{\beta}^{-1} \dot{\mathbf{s}}. \tag{23}$$

Combining equations (15), (17), and (23) results in:

$$\dot{V}_2 = -c_1 \mathbf{e}_1^T \mathbf{e}_1 - K||\mathbf{s}||_1 + \mathbf{s}^T(\mathbf{p} - \widehat{\mathbf{p}}) \tag{24}$$

$$\leq -c_1 \mathbf{e}_1^T \mathbf{e}_1 - K||\mathbf{s}||_1 + |\mathbf{s}^T(\mathbf{p} - \widehat{\mathbf{p}})|. \tag{25}$$

For the stability check, we employ Hölder's inequality:

$$\dot{V}_2 \leq -c_1 \mathbf{e}_1^T \mathbf{e}_1 - K||\mathbf{s}||_1 + ||\mathbf{s}||_1 ||(\mathbf{p} - \widehat{\mathbf{p}})||_\infty. \tag{26}$$

Considering the assumption in equation (19), we have:

$$\dot{V}_2 \leq -c_1 \mathbf{e}_1^T \mathbf{e}_1 - K||\mathbf{s}||_1 + ||\mathbf{s}||_1 \bar{p}_\infty. \tag{27}$$

Choosing $K = \bar{p}_\infty + \kappa$ with κ being a positive constant leads to:

$$\dot{V}_2 \leq -c_1 \mathbf{e}_1^T \mathbf{e}_1 - \kappa||\mathbf{s}||_1. \tag{28}$$

Hence, $\dot{V}_2 \leq 0$. This confirms the second control law is asymptotically stable.

New switching control design

The switching term $\boldsymbol{\tau}_{sw} = -K\text{sign}(\mathbf{s})$ is vital for ensuring the control input aligns with the target trajectory. However, the sign function's discontinuity often causes chattering, which hampers control performance (Slotine and Li, 1991). To address this, many continuous alternatives have been proposed (Nguyen and Tran, 2023a, 2023b). Here, we adopt the hyperbolic tangent function, resulting in the updated switching term:

$$\boldsymbol{\tau}_{sw} = -K\tanh(\mathbf{s}). \tag{29}$$

Insights on the RBFNN

Classical model

Typically, an RBFNN is organized into three distinct layers: an input layer, a hidden layer, and an output layer. To promote clarity and consistency in the description of RBFNNs' architecture, a concise notation is employed (Dinh et al., 2023; Kim et al., 2023; Liu, 2013):

$$M - E - O,$$

where

- M represents the dimension of each input vector;
- E denotes the number of nodes (neurons) in the hidden layer; and
- O indicates the number of output nodes per input vector.

For instance, an RBFNN with three input vectors of size 2, 5 hidden neurons, and 1 output neuron per input vector is described using the structure notation "$2-5-1$." Figure 1 provides a visual representation for this specific network configuration.

Let we consider an anisotropic RBFNN, where each node has distinct width parameters for each dimension of the input space. For an anisotropic RBFNN with I inputs in $\mathbb{R}^M$ and E nodes in its hidden layer, each neuron is determined by two key parameters: the center coordinates, which are defined as:

$$\mathbf{C} = \begin{bmatrix} c_{11} & c_{12} & \dots & c_{1E} \\ \dots & \dots & \dots & \dots \\ c_{M1} & c_{M2} & \dots & c_{ME} \end{bmatrix}, \tag{30}$$

and the widths (spreads), which can be expressed as:

$$\boldsymbol{\Sigma} = \begin{bmatrix} \sigma_{11} & \sigma_{12} & \dots & \sigma_{1E} \\ \dots & \dots & \dots & \dots \\ \sigma_{M1} & \sigma_{M2} & \dots & \sigma_{ME} \end{bmatrix}. \tag{31}$$

Moreover, the output of each node is calculated as:

$$u_i^{(k)} = \exp\left[-\frac{1}{2} \sum_{j=1}^{M} \frac{\left(x_j^{(k)} - c_{ji}\right)^2}{\sigma_{ji}^2} \right], \tag{32}$$

where

- $i \in [1, E]$, $j \in [1, M]$, and $k \in [1, I]$;
- $u_i^{(k)}$ is the output of the i-th node for the k-th input vector;
- $x_j^{(k)}$ is the j-th dimension of the k-th input vector;
- c_{ji} is the center coordinate of the i-th node in the j-th dimension; and
- σ_{ji} is the width parameter of the i-th node in the j-th dimension.

It is worth noting that $u_i^{(k)} \in [0, 1]$. In the end, the output produced by the RBFNN is:

$$y^{(k)} = \sum_{i=1}^{E} w_i^{(k)} u_i^{(k)}, \tag{33}$$

where

- $y^{(k)}$ is the output for the k-th input vector;
- $w_i^{(k)}$ is the weight associated with the i-th node for the k-th input vector; and
- $u_i^{(k)}$ is the output of the i-th node for the k-th input vector.

This paper develops a model to capture the dynamics of a real-world system with I input-output pairs:

$$\{(\mathbf{X}_1, \mathbf{Y}_1), \dots, (\mathbf{X}_i, \mathbf{Y}_i), \dots, (\mathbf{X}_I, \mathbf{Y}_I)\},$$

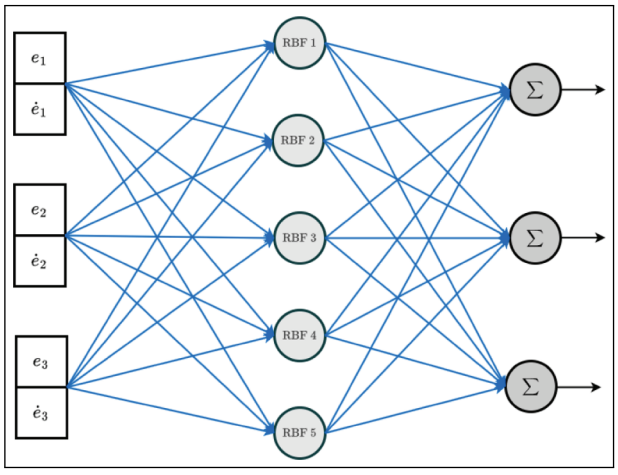


Figure 1. A "2-5-1" RBFNN configured with three input vectors.

where $\mathbf{X}_i$ is an input vector of size M, and $\mathbf{Y}_i$ denotes the measured outputs. The anisotropic RBFNN output is given by:

$$\hat{\mathbf{y}} = f(\mathbf{X}, \mathbf{P}),$$

with $\mathbf{X}$ as the input matrix and $\mathbf{P}$ encapsulating all K parameters, including centers, widths, and weights:

$$\mathbf{P} = \mathbf{C} \cup \mathbf{\Sigma} \cup \mathbf{W}.$$

The total hyperparameter count is $L = 2(M \times E) + I \times E$. As the model complexity increases with neurons and inputs, the rapid growth of L poses optimization challenges. We will propose strategies to reduce this hyperparameter count and simplify the model.

Pruning and optimizing strategies

Pruning strategies. To reduce the hyperparameter count L in the RBFNN, three simplification strategies are proposed.

First, assuming uniform width across dimensions, each hidden-layer node uses a scalar width σ instead of an M-dimensional vector width $\mathbf{\Sigma}$. This simplifies the neuron output computation to:

$$u_i^{(k)} = \exp\left[-\frac{1}{2}\sum_{j=1}^{M}\frac{\left(x_j^{(k)} - c_{ji}\right)^2}{\sigma_i^2}\right], \tag{34}$$

and reduces L to $L = M \times E + I \times E + E$.

Next, under width sharing between RBFs, all Gaussian functions share a common width σ. This further simplifies the neuron output to:

$$u_i^{(k)} = \exp\left[-\frac{1}{2}\sum_{j=1}^{M}\frac{\left(x_j^{(k)} - c_{ji}\right)^2}{\sigma^2}\right], \tag{35}$$

and lowers L to $M \times E + I \times E + 1$.

Finally, center sharing between RBFs is introduced, where all neurons not only share the same width σ but also a common center c throughout the input space. With this setup, the neuron's output is calculated as:

$$u_i^{(k)} = \exp\left[-\frac{1}{2}\sum_{j=1}^{M}\frac{\left(x_j^{(k)} - c\right)^2}{\sigma^2}\right], \tag{36}$$

and the hyperparameter count is further reduced to $L = M \times E + 2$.

Optimizing strategy: Clustering method for center and width initialization. To initialize the RBFNN effectively within our adaptive control scheme, clustering of error data is essential. To capture both spatial and temporal characteristics of the data, we employ the sequential nearest neighbor clustering (SNNC) algorithm, an advancement of the modified nearest neighbor clustering (MNNC) method (Zheng et al., 2021).

Before delving into the algorithm, we first examine the input to the RBFNN. At each time step k, the data point E_k is given by:

$$E_k = \left((e_{1k}, \dot{e}_{1k}), (e_{2k}, \dot{e}_{2k}), (e_{3k}, \dot{e}_{3k})\right), \tag{37}$$

where e_{ak} and $\dot{e}_{ak}$ represent the angular and velocity errors of each joint $a = \{1, 2, 3\}$, respectively. For simplicity, assuming similar characteristics among these values, we define a combined error e_k and a combined velocity error $\dot{e}_k$ as:

$$e_k = \frac{e_{1k} + e_{2k} + e_{3k}}{3}, \tag{38}$$

$$\dot{e}_k = \frac{\dot{e}_{1k} + \dot{e}_{2k} + \dot{e}_{3k}}{3}. \tag{39}$$

Thus, the data point E_k can be rewritten as:

$$E_k = (e_k, \dot{e}_k). \tag{40}$$

The algorithm begins by determining the thresholds $d_{e,0}^k$ and $d_{\dot{e},0}^k$ for clustering. Initially, data points are sorted along

the first dimension (e.g. e), and the nearest neighbors to the point E_k are identified as $E^k_{e,1}$ and $E^k_{e,2}$. The distances from E_k to these neighbors are calculated as:

$$d(E_k, E^k_{e,1}) = \sqrt{(e_{e,1} - e_k)^2 + (\dot{e}_{e,1} - \dot{e}_k)^2}, \tag{41}$$

$$d(E_k, E^k_{e,2}) = \sqrt{(e_{e,2} - e_k)^2 + (\dot{e}_{e,2} - \dot{e}_k)^2}. \tag{42}$$

This process is repeated for the second dimension (e.g. $\dot{e}$), with nearest neighbors $E^k_{\dot{e},1}$ and $E^k_{\dot{e},2}$. The corresponding distances are:

$$d(E_k, E^k_{\dot{e},1}) = \sqrt{(e_{\dot{e},1} - e_k)^2 + (\dot{e}_{\dot{e},1} - \dot{e}_k)^2}, \tag{43}$$

$$d(E_k, E^k_{\dot{e},2}) = \sqrt{(e_{\dot{e},2} - e_k)^2 + (\dot{e}_{\dot{e},2} - \dot{e}_k)^2}. \tag{44}$$

The thresholds $d^k_{e,0}$ and $d^k_{\dot{e},0}$ are then defined as the minimum distances:

$$d^k_{e,0} = \min\left(d(E_k, E^k_{e,1}), d(E_k, E^k_{e,2})\right), \tag{45}$$

$$d^k_{\dot{e},0} = \min\left(d(E_k, E^k_{\dot{e},1}), d(E_k, E^k_{\dot{e},2})\right). \tag{46}$$

Following this, the algorithm clusters data points by placing them in a temporal order and checking if the differences in both dimensions between E_k and its neighboring points are within these thresholds. A neighboring point E_j is considered a match if:

$$|e_j - e_k| \leq d^k_{e,0}, \tag{47}$$

$$|\dot{e}_j - \dot{e}_k| \leq d^k_{\dot{e},0}. \tag{48}$$

For points satisfying this condition, the distance to E_i is calculated as:

$$d(E_k, E_j) = \sqrt{(e_j - e_k)^2 + (\dot{e}_j - \dot{e}_k)^2}. \tag{49}$$

The closest neighbor is then determined by selecting the minimum of these distances:

$$d_k = \min\left(d(E_k, E_1), d(E_k, E_2), \cdots, d(E_k, E_j)\right). \tag{50}$$

This process helps form clusters. The pseudo-code for the SNNC algorithm is provided in Algorithm 1.

Algorithm 1 SNNC for RBFNN initialization.

Input: Data points $(E_1, \ldots, E_N)$, distance level H.
Output: Clustering result.

```
for each data point E_k do
    Calculate d_k of E_k;
end for
d_max ← max(d_1, d_2, ..., d_N);
d_min ← min(d_1, d_2, ..., d_N);
d_step ← (d_max + d_min) / H;
for i ← 1 to H do
    for each data point E_k do
        Find E_j s.t. ||E_j − E_k|| ≤ d_min + H × d_step;
        Construct cluster_k = (E_k, E_j);
    end for
    for each cluster_k do
        if cluster_k ∩ cluster_j ≠ ∅ then
            cluster_k = cluster_k + cluster_j;
        end if
    end for
end for
Merge small clusters into the nearest cluster;
return clusters.
```

Adaptive law design

RBFNN-based adaptive law

In theory, precise identification of model uncertainties and external disturbances, $\mathbf{p}$, is critical for ensuring system stability. However, achieving this in practice is highly challenging. To address this, an RBFNN-based adaptive law is incorporated into the controller to enhance estimation. Specifically, a compact "2-7-1" RBFNN structure is utilized to minimize computational demands, while an error-driven input mechanism improves adaptability. The uncertainty estimation is given by Liu (2017):

$$\mathbf{p} = \mathbf{W}^T \mathbf{H}(\mathbf{X}) + \varepsilon, \tag{51}$$

where

- $\mathbf{W}$ represents the optimal weights determined through training;
- $\mathbf{H}(\mathbf{X})$ denotes the output of neurons for the input $\mathbf{X}$; and
- ε is the modeling error.

For a 3-DoF robot arm, the input space includes three vectors:

$$\mathbf{x}_a = \begin{bmatrix} \mathbf{e}_a & \dot{\mathbf{e}}_a \end{bmatrix}^T, \tag{52}$$

$$\mathbf{x}_b = \begin{bmatrix} \mathbf{e}_b & \dot{\mathbf{e}}_b \end{bmatrix}^T, \tag{53}$$

$$\mathbf{x}_c = \begin{bmatrix} \mathbf{e}_c & \dot{\mathbf{e}}_c \end{bmatrix}^T, \tag{54}$$

where each error $\mathbf{e}_a$, $\mathbf{e}_b$, and $\mathbf{e}_c$ and its derivative represent deviations between the desired and actual joint positions and velocities. During training, the system must adapt to real-time disturbances using network-based estimation, defined as:

$$\widehat{\mathbf{p}} = \widehat{\mathbf{W}}^T \mathbf{H}, \tag{55}$$

where $\widehat{\mathbf{W}}$ is the estimated weight matrix. The adaptive law is designed as follows:

$$\dot{\widehat{\mathbf{W}}} = \eta \mathbf{H} \mathbf{s}^T, \tag{56}$$

where η is a positive constant and referred to as the learning rate. In addition, the adaptive process introduces a weight error defined as:

$$\widetilde{\mathbf{W}} = \mathbf{W} - \widehat{\mathbf{W}}. \tag{57}$$

Stability analysis

For the stability check, we choose the following Lyapunov function:

$$V_3 = V_2 + \frac{1}{2}\mathrm{tr}\left(\eta^{-1}\widetilde{\mathbf{W}}^T\widetilde{\mathbf{W}}\right). \tag{58}$$

where η is a positive constant. Taking the time derivative of V_3 gives:

$$\begin{aligned}\dot{V}_3 &= -c_1\mathbf{e}_1^T\mathbf{e}_1 - K||\mathbf{s}||_1 + \mathbf{s}^T(\mathbf{p}-\widehat{\mathbf{p}}) \\ &\quad + \mathrm{tr}\left(\eta^{-1}\widetilde{\mathbf{W}}^T\dot{\widetilde{\mathbf{W}}}\right) \qquad (59)\\ &= -c_1\mathbf{e}_1^T\mathbf{e}_1 - K||\mathbf{s}||_1 + \mathbf{s}^T\widetilde{\mathbf{W}}^T\mathbf{H} \\ &\quad + \mathrm{tr}\left(\eta^{-1}\widetilde{\mathbf{W}}^T\dot{\widetilde{\mathbf{W}}}\right) \qquad (60)\\ &= -c_1\mathbf{e}_1^T\mathbf{e}_1 - K||\mathbf{s}||_1 + \mathrm{tr}\left(\widetilde{\mathbf{W}}^T\mathbf{H}\mathbf{s}^T\right) \\ &\quad + \mathrm{tr}\left(\eta^{-1}\widetilde{\mathbf{W}}^T\dot{\widetilde{\mathbf{W}}}\right) \qquad (61)\\ &= -c_1\mathbf{e}_1^T\mathbf{e}_1 - K||\mathbf{s}||_1 \\ &\quad + \mathrm{tr}\left[\widetilde{\mathbf{W}}^T\left(\mathbf{H}\mathbf{s}^T + \eta^{-1}\dot{\widetilde{\mathbf{W}}}\right)\right]. \qquad (62)\end{aligned}$$

With $\dot{\widetilde{\mathbf{W}}} = -\dot{\widehat{\mathbf{W}}} = -\eta\mathbf{H}\mathbf{s}^T$, it follows that $\dot{V}_3 \leq 0$. This condition confirms the Lyapunov stability, ensuring the proposed control law is asymptotically stable.

Simulation setup and procedures

Setup

In this study, we use a manipulator modeled after the IRB1600 (ABB Robotics, 2024), a six-degree-of-freedom (6-DoF) industrial robot. To emphasize positioning accuracy over full-range articulation, the model is simplified to three degrees of freedom. This reduction allows focused analysis of control performance, reflecting industrial needs that prioritize precise spatial positioning over complex orientation control. The RBFNN structure is configured as "2-7-1" with three input vectors.

Evaluation metric

Evaluation metrics are critical in real-world applications for quantitatively assessing and comparing solutions. This study adopts the root mean squared error (RMSE) as the evaluation metric. Despite criticisms regarding its interpretability (Willmott et al., 2009) and its limitations as a measure of average error (Willmott and Matsuura, 2005), RMSE remains a widely used and accepted standard for model evaluation. The RMSE formula is as follows:

$$\mathrm{RMSE}\alpha = \sqrt{\frac{1}{n+1}\sum_{k=1}^{n}\left[\Delta\alpha(k)\right]^2}, \tag{63}$$

Table 2. Parameter values of the proposed control system.

Component	Parameter	Numerical value
Joint motor masses	m_1	10 kg
	m_2	8.0 kg
	m_3	5.0 kg
Link lengths	l_1	0.6 m
	l_2	0.8 m
	l_3	1.2 m
Sliding surface	γ	3
	σ	2
	λ	15
Controller	c_1	0.01
	η	0.01
	K	5

where

- α is the variable under analysis;
- $\Delta\alpha(k)$ denotes the k-th difference between actual and target values; and
- n is the total number of data points.

Simulation

The optimal control system parameters, which have been refined through extensive calibration, are listed in Table 2. These parameters were applied throughout all stages of the simulation process to ensure consistency.

Simulations and visualizations were conducted using MATLAB Simulink, with the control scheme illustrated in Figure 2. Outputs were compared between our proposed NTSMC-RBFNN, Adaptive Fuzzy-SMC (Amer et al., 2011), and SMC-neural network (NN) (Truong et al., 2019) control methods. The evaluation metric used was RMSE, as introduced earlier. A 10-second simulation duration was selected, as all methods converge within 4 seconds.

Simulations are conducted under two conditions: ideal and disturbed environments. While a sine wave reference is commonly used for smooth control in industrial settings, a square wave is also considered to evaluate performance under abrupt changes. The initial state is set to the home position, defined as $\mathbf{q}_0 = \begin{bmatrix}0 & 0 & 0\end{bmatrix}^T$. The ideal sine wave reference is given by $\mathbf{q}_d = \begin{bmatrix}0.5\cdot\sin(t) & 0.5\cdot\sin(t) & 0.5\cdot\sin(t)\end{bmatrix}^T$, while the ideal square wave reference is defined as $\mathbf{q}_d = \begin{bmatrix}0.5\cdot\mathrm{square}(t) & 0.5\cdot\mathrm{square}(t) & 0.5\cdot\mathrm{square}(t)\end{bmatrix}^T$.

The robustness of the control system is evaluated using a disturbed reference signal that reflects real-world conditions, including friction, external disturbances, power fluctuations, and physical obstacles. The disturbance model consists of two main components. First, additive noise is defined as $\mathbf{n} = \begin{bmatrix}0.025\cdot\sin(t) & 0.025\cdot\sin(t) & 0.025\cdot\sin(t)\end{bmatrix}^T$. Second, friction effects include a velocity-dependent term $\mathbf{f}_v = 0.025\cdot$

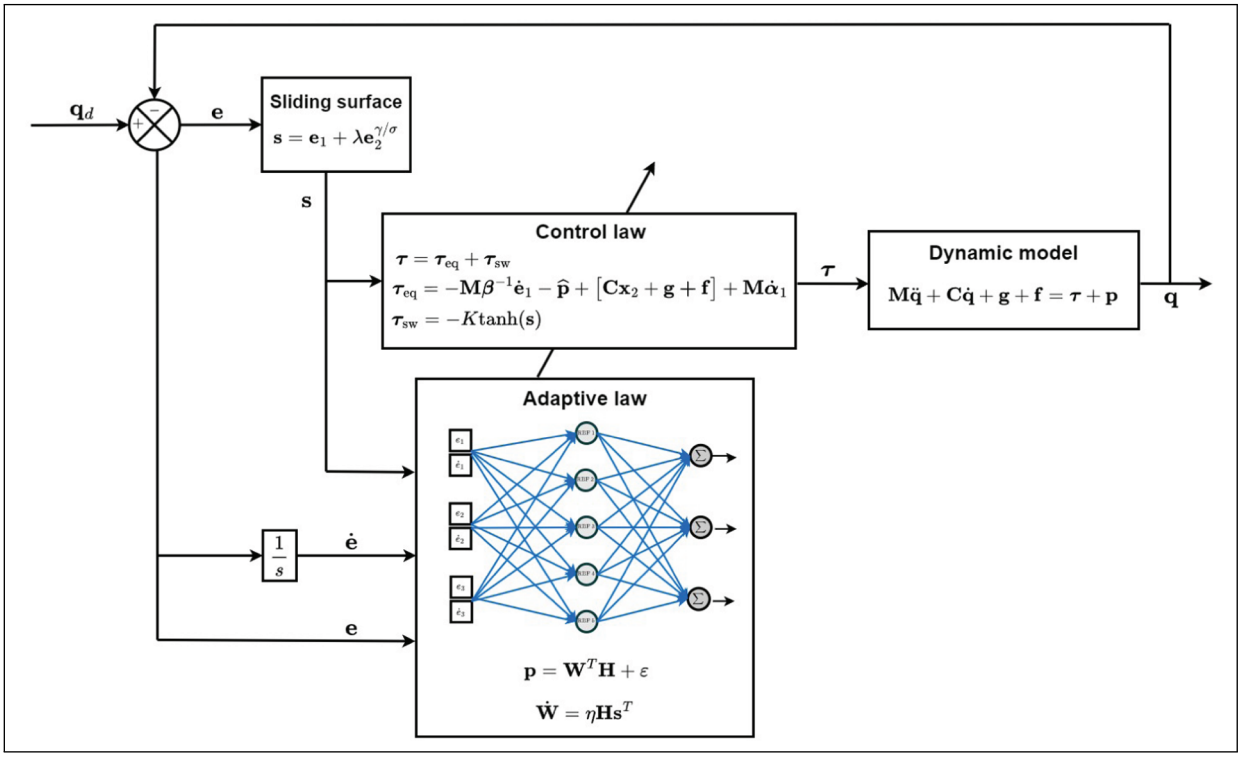


Figure 2. Schematic of the proposed control system.

Table 3. Centroid coordinate values.

Cluster	Dimension 1	Dimension 2
1	0.014	0.910
2	0.030	0.530
3	0.030	0.130
4	0.030	−0.300
5	0.010	−0.630
6	−0.010	−0.260
7	−0.010	0.010
Average	**0.013**	**0.056**

$\text{sign}(\dot{\mathbf{q}})$ and a directional component $\mathbf{f}_d = 0.025 \cdot \text{diag}(0.025) \cdot \text{sign}(\dot{\mathbf{q}})$. Additional details of the friction model are provided in the Appendix A.

Results and discussions

Clustering results

The clustering analysis revealed the presence of seven distinct clusters after 460 iterations. The centroid coordinates for these clusters are displayed in Table 3. Assuming width and center sharing among RBFs, the optimal center coordinates for the RNFNN were determined by averaging the average first and second dimensions of the centroid coordinates, resulting in $c = 0.035$. Following that, the optimal width (σ) was computed using Haykin's (1999) formula:

$$\sigma = \frac{d_{\max}}{\sqrt{2M}}, \tag{64}$$

where M represents the number of centroids (7 in this case) and $d_{\max}$ denotes the maximum distance between any two centroids. This calculation yielded an optimal width of $\sigma = 0.412$. These parameter choices significantly improved the precision and robustness of the control strategy, as detailed in the following sections.

Tracking control results

This section focuses on evaluating the first link of the manipulator due to its critical role in the robot's overall dynamics. The behavior of the remaining links is provided in the Appendix A. As the base joint, it supports the highest load and generates the greatest torque, particularly during abrupt motion changes. Ensuring precise and safe control of this core joint helps maintain stability and accuracy across the entire system. Moreover, its contribution to large-scale motion makes it an effective benchmark for assessing the controller's robustness under disturbances and abrupt transitions.

The proposed control method demonstrates superior performance, outperforming both the SMC-NN and Adaptive Fuzzy-SMC techniques under ideal and disturbed conditions. As shown in Table 4, it achieves an RMSE of 0.000202 in ideal scenarios, approximately half that of Adaptive Fuzzy-SMC (0.000477) and significantly better than SMC-NN (0.015168). This high precision is maintained even in disturbed environments, where the proposed method attains an RMSE of

0.002634, nearly six times lower than the 0.011748 recorded for Adaptive Fuzzy-SMC. This performance is attributed to the newly introduced backstepping technique for adaptive sliding surface construction, which ensures smooth integration between error dynamics and control law design. As a result, the controller enhances trajectory tracking accuracy while maintaining stability and robustness under model uncertainties and external disturbances.

Table 5 highlights the fast convergence of the proposed control method. Under ideal conditions, it stabilizes the system in 0.037533 seconds, significantly faster than the Adaptive Fuzzy-PID-SMC method, which requires 0.388067 seconds. In disturbed environments, the proposed method maintains its advantage with an average convergence time of 0.280800 seconds, compared to 1.146867 and 2.488133 seconds for the other two approaches. While Fuzzy-SMC and SMC-NN offer balanced overall performance, these results emphasize the proposed method's ability to deliver not only precision and stability but also rapid response across varying conditions.

Figures 3 and 4 demonstrate the superior noise robustness of the proposed method and its ability to respond almost instantly to the desired position. While SMC-NN exhibits less chattering than Fuzzy-PID-SMC, it still shows small spikes in the control input and requires an initial torque of approximately 130 Nm to position the arm. In contrast, the proposed method achieves the same task with a startup torque of less than 50 Nm. This key result highlights the controller's effectiveness in minimizing control effort while maintaining high precision under both ideal and disturbed conditions.

Similar patterns are observed in scenarios involving abrupt changes in robot motion. To assess the controller's effectiveness and robustness under such conditions, a square wave reference is used. As shown in Figures 5 and 6, the proposed method maintains stability, while Fuzzy-PID-SMC and SMC-NN exhibit multiple spikes in control input, reaching up to 220 Nm, levels that may lead to long-term damage to the physical system. Although both comparative methods adapt to disturbances in square wave scenarios, they show excessive fluctuations, particularly during sharp transitions. In contrast, the proposed method outperforms both controllers by ensuring smooth trajectory tracking, fast response, and efficient power usage.

The success of the proposed control method stems from several key design features. First, the modified equivalent control and Tanh-based switching law effectively eliminate the chattering commonly observed in traditional sliding mode controllers, while ensuring precise trajectory tracking. Second, the compact RBFNN structure, combined with an effective parameter initialization strategy, enables accurate estimation of internal and external disturbances in near real time. Third, system stability is reinforced by a well-tuned learning rate η in the adaptive law. These enhancements address the limitations of fixed-gain methods, such as large ripples and high sensitivity to disturbances.

Computational considerations

The RBFNN-based adaptive module is designed for computational efficiency and minimal parameterization. It is implemented in Simulink using an Embedded MATLAB Function block, with online adaptation performed through a linear-in-parameter update law. The model runs under a fixed-step solver with a sampling time of 0.2 milliseconds. Simulink Profiler reports an average update time of approximately 0.085 milliseconds per step, confirming that adaptation completes well within the control cycle. Offline initialization using improved k-means clustering for center placement takes 10.2 seconds on an Intel Core i7-9750H processor. This compact architecture enables fast adaptation, low computational overhead, and seamless integration into high-frequency control loops without requiring high-end hardware.

Joint torque considerations

To ensure realistic control behavior, saturation limits are incorporated into the simulation using Simulink saturation blocks. These constraints keep the control input within the actuator's operating range, with a maximum torque of approximately 250 Nm. As shown in the Appendix A, the proposed method not only maintains inputs within these bounds but also achieves the optimal torque required for operation. This approach ensures all control actions remain physically feasible and enhances both the fidelity of the simulation and its relevance to real-world robotic applications.

Non-periodic disturbances considerations

The disturbance model primarily addresses periodic and friction-based disturbances, representing predictable effects such as machinery vibrations, power fluctuations, and repetitive task interactions. Non-periodic disturbances, including impulse forces, are not explicitly modeled, as their impact on adaptive control systems is typically transient and diminishes over time. As noted in Tenney and Tomizuka (1996), repetitive controllers gradually learn to cancel recurring disturbances, while one-time disturbances may cause only temporary deviations before the system recovers. In the proposed NTSMC-RBFNN framework, the adaptive term, supported by a compact RBFNN structure, provides real-time compensation for both periodic and transient non-periodic uncertainties without relying on iterative learning or disturbance history. Overall, this study focuses on periodic disturbances to evaluate the controller's effectiveness under long-lasting conditions. Future work will investigate performance under a wider range of complex non-periodic disturbances to further assess robustness.

Practicality considerations

Although the proposed scheme has strong theoretical potential, most related studies remain limited to simulations and offer only conceptual guidance for practical implementation. In contrast, two studies closely related to our NTSMC-RBFNN framework present complete controller designs with simulation results that address real-world nonlinearities and uncertainties in robotic systems, aiming at industrial applications. The first applies an RBFNN-based Nonsingular Fast Terminal SMC to an insulator-cleaning robot operating in high-voltage

Table 4. RMSE for ideal and disturbed sine wave cases.

Type	Link	Proposed	Fuzzy-SMC	SMC-NN
Ideal	Link1	0.000155	0.000646	0.003352
	Link2	0.000279	0.000492	0.029312
	Link3	0.000172	0.000293	0.012840
	Average	**0.000202**	**0.000477**	**0.015168**
Disturbed	Link1	0.001909	0.009535	0.033496
	Link2	0.004341	0.017284	0.429258
	Link3	0.001653	0.008427	0.212713
	Average	**0.002634**	**0.011748**	**0.225156**

Table 5. Convergence time for ideal and disturbed sine wave cases.

Type	Link	Proposed	Fuzzy-SMC	SMC-NN
Ideal	Link1	0.034800	0.565000	1.565000
	Link2	0.040200	0.360800	1.409200
	Link3	0.037600	0.240400	1.755400
	Average	**0.037533**	**0.388067**	**1.576533**
Disturbed	Link1	0.171000	1.085200	2.256200
	Link2	0.359000	1.182400	1.959600
	Link3	0.310400	1.173000	3.248600
	Average	**0.280800**	**1.146867**	**2.488133**

substations (Jin et al., 2024). It accounts for uneven terrain and disturbances such as variable contact forces and surface irregularities. A 2-DoF dynamic model, derived using Lagrangian mechanics, captures the robot's longitudinal motion. Simulation results demonstrate fast, stable, and precise tracking suitable for autonomous maintenance. The second study focuses on a 5-DoF upper-limb rehabilitation exoskeleton (Li et al., 2024), where strict timing and bounded tracking errors are critical for patient safety and therapeutic outcomes. A Practical Predefined-Time NTSMC framework, combined with RBFNN-based disturbance compensation, guarantees convergence within a specified time, regardless of initial conditions. This ensures consistent and safe rehabilitation motion. Together with our findings, these studies demonstrate that NTSMC-RBFNN is both theoretically reliable and practically effective in high-safety applications such as power system maintenance and physical rehabilitation.

Extension to other robotic systems

The proposed NTSMC, Backstepping, and RBFNN-based control framework, originally developed for a 3-DoF serial manipulator, is structurally general and can be extended to planar and spatial parallel robots with appropriate system-specific modifications (Lai et al., 2018; Wang et al., 2015). Parallel robots differ from serial manipulators in that they exhibit closed-loop kinematics, internal constraint forces, and actuator redundancy (Murray et al., 1994). These characteristics require constraint-consistent dynamic modeling and specialized torque allocation strategies, often using Lagrange multipliers or reduced coordinate methods. Despite these modeling differences, the key components of the controller retain their theoretical validity when appropriately adapted to such architectures.

In planar parallel manipulators, such as 2-DoF RPR and 3-RRR mechanisms, actuator motions are coupled due to geometric loop closures. Wang et al. (2009) addressed this challenge by designing an adaptive backstepping controller for a planar 2-DoF parallel platform, achieving asymptotic task-space tracking in the presence of dynamic and parametric uncertainties. Their method used constraint-aware dynamics and Lyapunov-based adaptation, demonstrating compatibility with backstepping architectures. Similarly, Van Nguyen and Ha (2018) applied RBFNN-augmented SMC to a rotary Stewart platform. Their approach effectively handled nonlinear uncertainties and interdependent actuation, confirming that neural adaptation and robustness techniques are compatible with constrained robot configurations.

Spatial parallel robots, such as the 6-DoF 6-UPS Stewart platform, add complexity due to coupled multi-axis motion and higher-order constraints. Wang et al. (2019) applied an adaptive proportional–derivative (PD) controller with RBFNN-based disturbance estimation on a wire-driven 6-DoF parallel manipulator. Their approach improved both transient performance and steady-state tracking by compensating lumped dynamic uncertainties in real time. In a related study, Qi et al. (2025)

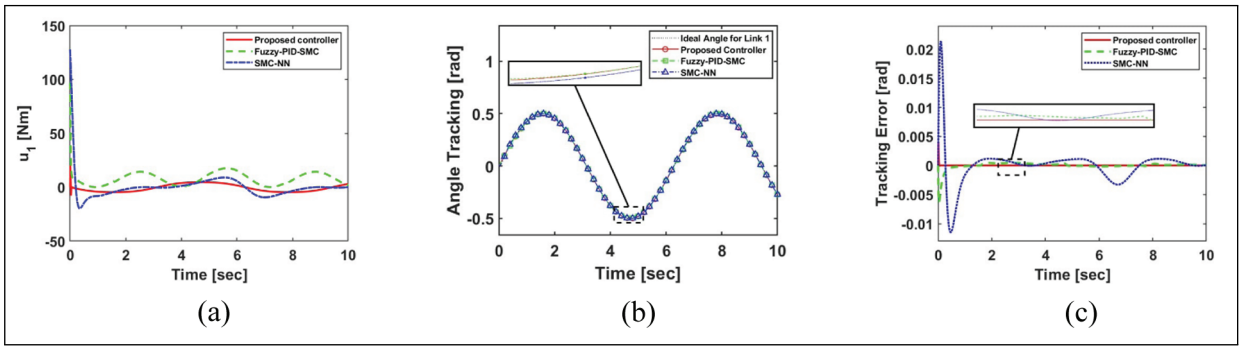


Figure 3. Tracking performance of link 1 with ideal sine wave: (a) control input, (b) angle tracking, and (c) tracking error.

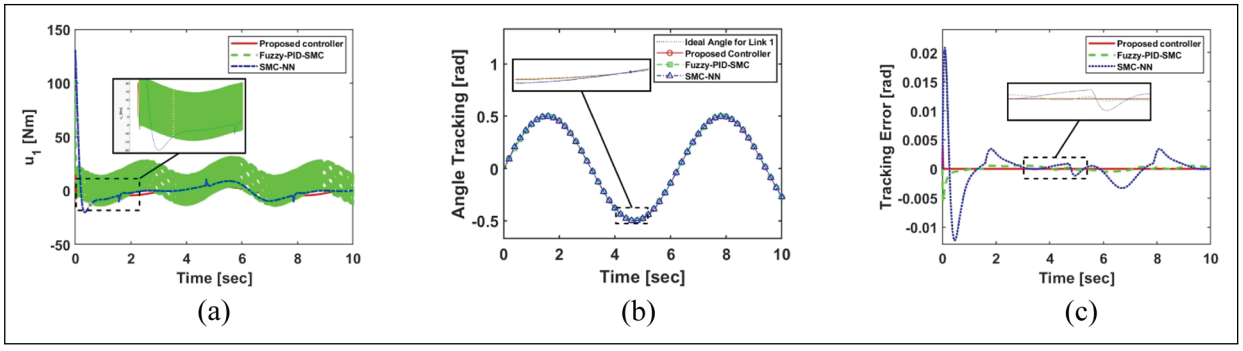


Figure 4. Tracking performance of link 1 with disturbed sine wave: (a) control input, (b) angle tracking, and (c) tracking error.

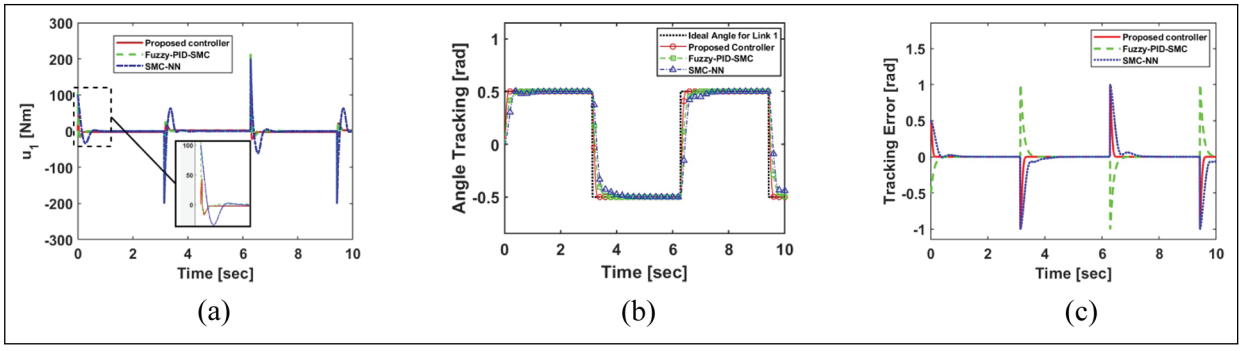


Figure 5. Tracking performance of link 1 with ideal square wave: (a) control input, (b) angle tracking, and (c) tracking error.

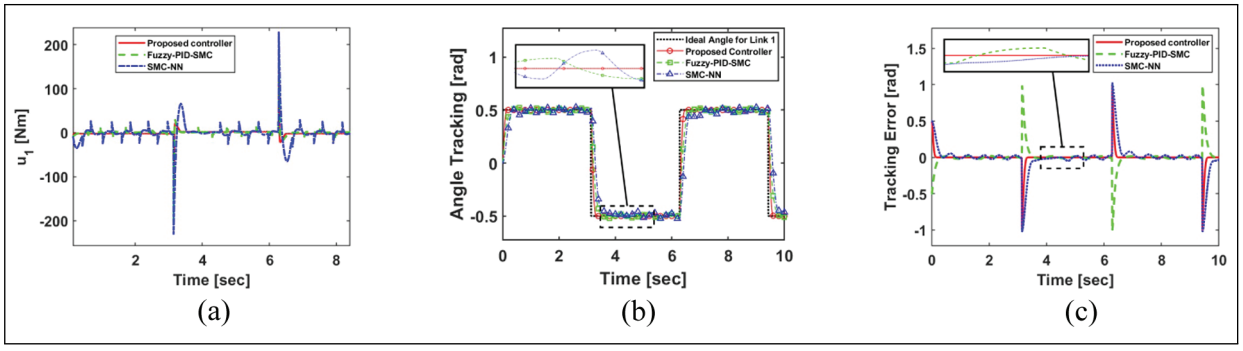


Figure 6. Tracking performance of link 1 with disturbed square wave: (a) control input, (b) angle tracking, and (c) tracking error.

designed an adaptive visual servoing controller for a Gough–Stewart platform, using RBFNNs to estimate load-dependent disturbances and joint friction. Their system preserved stability and precise pose regulation under unknown external forces. These implementations confirm that neural adaptive estimation and robust control schemes are viable for spatial parallel mechanisms with constrained dynamics and redundancy.

To extend the proposed framework, four primary modifications are required. First, dynamic equations must be reformulated using constraint-consistent methods, such as Lagrange multipliers or projection-based reduction (Murray et al., 1994). Second, the sliding surface should be defined in task space and may include constraint force components or reaction feedback, especially when full-state information is not available (Mistry and Righetti, 2012). Third, in the backstepping stage, the mapping from virtual control to actuator torque must be handled via the Jacobian transpose or Moore–Penrose pseudoinverse, accounting for redundancy and singularity conditions (Wang, 2022). Fourth, the RBFNN architecture must be adapted to produce vector-valued outputs capable of approximating coupled, time-varying disturbances across multiple degrees of freedom (Jayasudha et al., 2023). These modifications preserve the core guarantees of the original design, including finite-time convergence, robustness to bounded uncertainties, and mitigation of chattering while enabling application to both planar and spatial parallel robotic systems.

Conclusion

This paper proposed an adaptive control framework for a 3-DoF industrial robotic manipulator, integrating NTSMC, backstepping design, and a clustering-based RBFNN. The controller was developed to address limitations in existing methods related to chattering, convergence delay, and poor adaptability to nonlinear disturbances. A clustering mechanism was introduced to efficiently initialize RBFNN parameters, enhancing learning effectiveness without manual tuning. Simulation results under both ideal and disturbed conditions showed that the proposed method consistently outperformed Fuzzy-SMC and SMC-NN baselines in tracking accuracy, convergence speed, and robustness across diverse input signals. Notably, the controller achieved high-precision tracking while eliminating the trade-off between chattering suppression and trajectory smoothness.

The advantages of the proposed approach stem from its modular yet synergistic design: NTSMC provides finite-time convergence and robustness to matched uncertainties, backstepping offers structured error regulation across system states, and the clustering-enhanced RBFNN ensures fast and effective adaptation to changing dynamics. This combination results in smooth, stable, and precise control even under sudden input changes or external disturbances. Nonetheless, the method requires full-state feedback and introduces moderate computational complexity due to online neural adaptation, which may limit deployment in resource-constrained or high-DoF systems. Future work will focus on reducing computational demands, extending applicability to systems with partial state information, and experimentally validating the controller on planar and spatial parallel robots. These efforts aim to strengthen the framework's readiness for deployment in complex industrial environments where high precision and adaptability are essential.

Acknowledgements

The authors express sincere gratitude to Dr Nguyen Hai Nguyen (Hann) from RMIT University Vietnam for his valuable feedback.

Ethical considerations

This article does not contain any studies with human or animal participants.

Consent to participate

There are no human participants in this article, and informed consent is not required.

Consent for publication

Not applicable.

Declaration of conflicting interest

The authors declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The authors received no financial support for the research, authorship, and/or publication of this article.

ORCID iDs

Loc Nguyen https://orcid.org/0009-0003-4254-0750
Thanh Tran https://orcid.org/0000-0001-5405-2963

Data availability statement

Data available on request from the authors.

References

ABB Robotics (2024) Irb 1600 datasheet. Available at: https://library.e.abb.com/public/d2aab84b6958416689aaac343a6b0285/IRB+1600+datasheet_20240726.pdf?x-sign=tLgXiFW79i3IHVHKsGgvpnNAtV%2fMiPbZ8UEuur6q5SWvAN3vK5yiafWoJH%2feHKyq

Amer AF, Sallam EA and Elawady WM (2011) Adaptive fuzzy sliding mode control using supervisory fuzzy control for 3 DOF planar robot manipulators. *Applied Soft Computing* 11(8): 4943–4953.

Capone A and Hirche S (2019) Backstepping for partially unknown nonlinear systems using Gaussian processes. *IEEE Control Systems Letters* 3(2): 416–421.

Dinh VV, Pham BL, Nguyen VT, et al. (2023) Adaptive radial-basis function neural network control of a pneumatic actuator. In: Nguyen TDL, Verdú E, Le AN, et al. (eds) *Intelligent Systems and Networks*. Singapore: Springer, pp. 254–262.

Duan J, Zhang H, Zhang Q, et al. (2023) Research on neural network terminal sliding mode control of robotic arms based on novel reaching law and improved salp swarm algorithm. *Actuators* 12(12): 464.

Furlani EP (2022) *Engineering Systems Dynamics and Controls: Fundamentals Through Simulations*. Victoria, BC, Canada: BCcampus.

Hagan MT, Demuth HB, Beale MH, et al. (2014) *Neural Network Design*. Stillwater, OK: Martin Hagan.

Haykin S (1999) *Neural Networks: A Comprehensive Foundation*. International edition. Hoboken, NJ: Prentice Hall.

Ibarra J, Márquez R and Bernal M (2023) Overcoming backstepping limitations via a novel MIMO non-affine-in-control convex optimization technique. *Journal of the Franklin Institute* 360(12): 8320–8338.

Jayasudha V, Deepa N and Devi T (2023) Effective model for improving symptoms-based disease prediction by BiMM - BERT algorithm. In: *2023 international conference on computer communication and informatics (ICCCI)*, Coimbatore, India, 23–25 January, pp. 1–7. New York: IEEE.

Jin J, Shen X, Zhu Q, et al. (2024) Neural network-based terminal sliding mode control for insulator cleaning robots. In: *2024 IEEE 22nd international conference on industrial informatics (INDIN)*, Beijing, China, 18–20 August, pp. 1–6. New York: IEEE.

Khalil H (2002) *Nonlinear Systems*. Pearson Education. Hoboken, NJ: Prentice Hall.

Kim SJ, Jin M and Suh JH (2023) A study on the design of error-based adaptive robust RBF neural network back-stepping controller for 2-DOF snake robot's head. *IEEE Access* 11: 23146–23156.

Lai CY, Villacis Chavez DE and Ding S (2018) Transformable parallel-serial manipulator for robotic machining. *The International Journal of Advanced Manufacturing Technology* 97(5): 2987–2996.

Li J, Wang Q and Fang Y (2024) Adaptive neural network-based practical predefined-time nonsingular terminal sliding mode control for upper limb rehabilitation robots. *Communications in Nonlinear Science and Numerical Simulation* 136: 108085.

Liu J (2013) *Radial Basis Function (RBF) Neural Network Control for Mechanical Systems*. Berlin and Heidelberg: Springer.

Liu J (2017) *Intelligent Control Design and MATLAB Simulation*. Singapore: Springer.

Lynch KM and Park FC (2017) *Modern Robotics: Mechanics, Planning, and Control*. Cambridge: Cambridge University Press.

Mistry M and Righetti L (2012) Operational space control of constrained and underactuated systems. *Robotics: Science and systems* 7: 225–232.

Murray RM, Li Z and Sastry SS (1994) *A Mathematical Introduction to Robotic Manipulation*. CRC Press.

Nguyen LPT and Tran TT (2023a) Design of a novel switching controller for an industrial crane system. In: *2023 international symposium on electrical and electronics engineering (ISEE)*, Ho Chi Minh, Vietnam, 19–20 October, pp. 115–120. New York: IEEE.

Nguyen LPT and Tran TT (2023b) A family of sliding mode controllers: Analysis, design and comparison. In: *2023 international conference on system science and engineering (ICSSE)*, Ho Chi Minh, Vietnam, 27–28 July, pp. 462–467. New York: IEEE.

Qi J, Song S, Zhao Y, et al. (2025) NN-based visual servoing compensation control of a Gough–Stewart platform with uncertain load. *Scientific Reports* 15(1): 15500.

Slotine J and Li W (1991) *Applied Nonlinear Control*. Hoboken, NJ: Prentice Hall.

Su B (2025) Tracking control of robotic manipulator end-effector trajectory based on robust sliding mode method. *PLoS ONE* 20(4): e0320118.

Tenney JA and Tomizuka M (1996) Effects of non-periodic disturbances on repetitive control systems. *IFAC Proceedings Volumes* 29(1): 1452–1457.

Truong HVA, Tran DT and Ahn KK (2019) A neural network based sliding mode control for tracking performance with parameters variation of a 3-DOF manipulator. *Applied Sciences* 9(10): 2023.

Vachkov G, Christova N and Valova M (2014) Optimization strategies for tuning the parameters of radial basis functions network models. In: *2014 4th international conference on simulation and modeling methodologies, technologies and applications (SIMULTECH)*, Vienna, 28–30 August, pp. 443–450. New York: IEEE.

Vaidyanathan S and Azar A (2020) *Backstepping Control of Nonlinear Dynamical Systems*. Advances in Nonlinear Dynamical Systems and Robotics (ANDC). Cambridge, MA: Academic Press.

Van Nguyen T and Ha C (2018) RBF neural network adaptive sliding mode control of rotary Stewart platform. In: Huang DS, Gromiha MM, Han K, et al. (eds) *Intelligent Computing Methodologies*. Cham: Springer, pp. 149–162.

Wang L, Lu Z, Liu X, et al. (2009) Adaptive control of a parallel robot via backstepping technique. *International Journal of Systems, Control and Communications* 1(3): 312–324.

Wang Y, Lin Q, Wang X, et al. (2019) Adaptive PD control based on RBF neural network for a wire-driven parallel robot and prototype experiments. *Mathematical Problems in Engineering* 2019(1): 6478506.

Wang Y, Smith C, Karayiannidis Y, et al. (2015) Cooperative control of a serial-to-parallel structure using a virtual kinematic chain in a mobile dual-arm manipulation application. In: *2015 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, Hamburg, 28 September–2 October, pp. 2372–2379. New York: IEEE.

Wang Z (2022) *Modern Robotics: Theory and Practice*. Clemson, SC: Clemson University Press.

Willmott CJ and Matsuura K (2005) Advantages of the mean absolute error (MAE) over the root mean square error (RMSE) in assessing average model performance. *Climate Research* 30(1): 79–82.

Willmott CJ, Matsuura K and Robeson SM (2009) Ambiguities inherent in sums-of-squares-based error statistics. *Atmospheric Environment* 43(3): 749–752.

Xia X, Jia Y, Wang X, et al. (2022) Neural-network-based terminal sliding mode control of space robot actuated by control moment Gyros. *Aerospace* 9(11): 730.

Xu Z, Huang W, Li Z, et al. (2021) Nonlinear nonsingular fast terminal sliding mode control using deep deterministic policy gradient. *Applied Sciences* 11(10): 4685.

Yuan T, Zhang C, Yi F, et al. (2024) RBFNN-based adaptive integral sliding mode feedback and feedforward control for a lower limb exoskeleton robot. *Electronics* 13(6): 1043.

Zheng D, Jung W and Kim S (2021) RBFNN design based on modified nearest neighbor clustering algorithm for path tracking control. *Sensors* 21(24): 8349.

Zhu D, Huang P, Du B, et al. (2023) Adaptive nonsingular terminal sliding mode control of robot manipulator based on contour error compensation. *Scientific Reports* 13(1): 330.

Appendix A

Manipulator's forward kinematics

For the 3-DoF articulated robot arm shown in Figure 7, the Denavit–Hartenberg (DH) parameters are presented in Table 6, specifying the link parameters essential for forward kinematics analysis.

Using the DH parameters, the forward kinematics of the end effector relative to the base frame is determined by computing the transformation matrix $\mathbf{T}_3^0$. From this matrix, the coordinates of the end effector with respect to the base frame are expressed as:

$$\begin{aligned} p_x &= c_1(c_2 l_2 + c_{23} l_3), \\ p_y &= s_1(c_2 l_2 + c_{23} l_3), \\ p_z &= l_1 + l_2 s_2 + l_3 s_{23}, \end{aligned}$$

where

- $c_1 = \cos(q_1)$;
- $c_2 = \cos(q_2)$;
- $s_1 = \sin(s_1)$;
- $c_{23} = \cos(q_2 + q_3)$;
- $s_{23} = \sin(q_2 + q_3)$.

Manipulator's dynamics

Inertia matrices.

$$\mathbf{I_a} = \begin{bmatrix} 0.85277 & -0.06561 & 0.26963 \\ -0.06561 & 1.37516 & -0.09551 \\ 0.26963 & -0.09551 & 0.96321 \end{bmatrix},$$

$$\mathbf{I_b} = \begin{bmatrix} 0.85277 & -0.06561 & 0.26963 \\ -0.06561 & 1.37516 & -0.09551 \\ 0.26963 & -0.09551 & 0.96321 \end{bmatrix},$$

$$\mathbf{I_c} = \begin{bmatrix} 0.11546 & -0.00738 & 0.00193 \\ -0.00738 & 0.26568 & -0.00874 \\ 0.00193 & -0.00874 & 0.26790 \end{bmatrix}.$$

Mass matrix $\mathbf{M}(\mathbf{q})$.

$$\mathbf{M}(\mathbf{q}) = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix},$$

Table 6. DH parameters of the manipulator considered in this study.

Joint	q_i	d_i	a_i	α_i
Joint 1	q_1	l_1	0	$\frac{\pi}{2}$
Joint 2	q_2	0	l_2	0
Joint 3	q_3	0	l_3	0

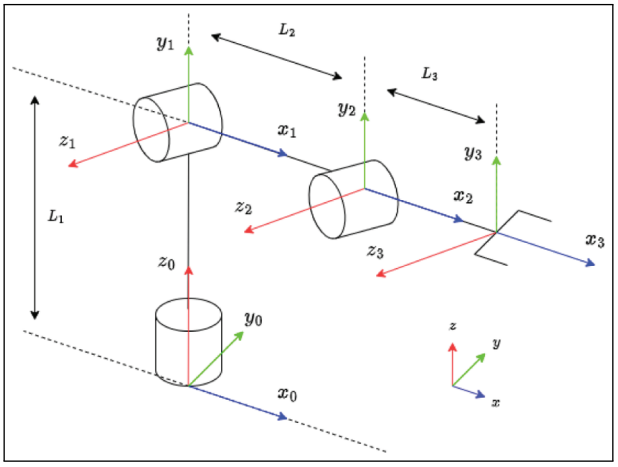


Figure 7. Kinematic model of the manipulator considered in this study.

where

- $M_{11} = I_{a22} + c_2\left(I_{b22}c_2 + I_{b12}s_2\right) + c_{23}\left(I_{c22}c_{23} + I_{c12}s_{23}\right) + s_2\left(I_{b21}c_2 + I_{b11}s_2\right) + s_{23}\left(I_{c21}c_{23} + I_{c11}s_{23}\right) + m_3\left(c_2 l_2 + c_{23} l_3\right)^2 - l_2^2 m_2\left(s_2^2 - 1\right)$;
- $M_{12} = I_{b23}c_2 + I_{c23}c_{23} + I_{b13}s_2 + I_{c13}s_{23}$;
- $M_{13} = I_{c23}c_{23} + I_{c13}s_{23}$;
- $M_{21} = I_{b32}c_2 + I_{c32}c_{23} + I_{b31}s_2 + I_{c31}s_{23}$;
- $M_{22} = I_{b33} + I_{c33} + l_2^2 m_2 + l_2^2 m_3 + l_3^2 m_3 + 2c_3 l_2 l_3 m_3$;
- $M_{23} = m_3 l_3^2 + c_3 l_2 m_3 l_3 + I_{c33}$;
- $M_{31} = I_{c32}c_{23} + I_{c31}s_{23}$;
- $M_{32} = m_3 l_3^2 + c_3 l_2 m_3 l_3 + I_{c33}$;
- $M_{33} = m_3 l_3^2 + I_{c33}$.

Coriolis and centrifugal matrix $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$.

$$\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) = \begin{bmatrix} C_{11} & C_{12} & C_{13} \\ C_{21} & C_{22} & C_{23} \\ C_{31} & C_{32} & C_{33} \end{bmatrix},$$

where

- $C_{11} = \frac{1}{2}\left[I_{b12}\dot{q}_2\cos(2q_2) + I_{b21}\dot{q}_2\cos(2q_2) + I_{b11}\dot{q}_2\sin(2q_2) - I_{b22}\dot{q}_2\sin(2q_2) + I_{c12}\dot{q}_2\cos(2q_2 + 2q_3) + I_{c12}\dot{q}_3\cos(2q_2 + 2q_3) + I_{c21}\dot{q}_2\cos(2q_2 + 2q_3) + I_{c21}\dot{q}_3\cos(2q_2 + 2q_3) + I_{c11}\dot{q}_2\sin(2q_2 + 2q_3) + I_{c11}\dot{q}_3\sin(2q_2 + 2q_3) - I_{c22}\dot{q}_2\sin(2q_2 + 2q_3) - I_{c22}\dot{q}_3\sin(2q_2 + 2q_3) - \dot{q}_2 l_2^2 m_2\sin(2q_2) - \dot{q}_2 l_2^2 m_3\sin(2q_2)\right]$

$$
\begin{aligned}
&- \dot{q}_2 l_3^2 m_3 \sin(2q_2 + 2q_3) - \dot{q}_3 l_3^2 m_3 \sin(2q_2 \\
&+ 2q_3) - 2\dot{q}_2 l_2 l_3 m_3 \sin(2q_2 + q_3) \\
&- \dot{q}_3 l_2 l_3 m_3 \sin(2q_2 + q_3) \\
&- \dot{q}_3 l_2 l_3 m_3 \sin(q_3)\big];
\end{aligned}
$$

- $$
\begin{aligned}
C_{12} = \frac{1}{2}\big[&I_{b12}\dot{q}_1 \cos(2q_2) + I_{b21}\dot{q}_1 \cos(2q_2) \\
&+ I_{b11}\dot{q}_1 \sin(2q_2) - I_{b22}\dot{q}_1 \sin(2q_2) \\
&+ I_{c12}\dot{q}_1 \cos(2q_2 + 2q_3) + I_{c21}\dot{q}_1 \cos(2q_2 \\
&+ 2q_3) + I_{c11}\dot{q}_1 \sin(2q_2 + 2q_3) \\
&- I_{c22}\dot{q}_1 \sin(2q_2 + 2q_3) + I_{c13}\dot{q}_2 \cos(q_2 \\
&+ q_3) + I_{c13}\dot{q}_3 \cos(q_2 + q_3) \\
&- I_{c23}\dot{q}_2 \sin(q_2 + q_3) - I_{c23}\dot{q}_3 \sin(q_2 + q_3) \\
&+ I_{b13}\dot{q}_2 \cos(q_2) - I_{b23}\dot{q}_2 \sin(q_2) \\
&- \dot{q}_1 l_2^2 m_2 \sin(2q_2) - \dot{q}_1 l_2^2 m_3 \sin(2q_2) \\
&- \dot{q}_1 l_3^2 m_3 \sin(2q_2 + 2q_3) \\
&- 2\dot{q}_1 l_2 l_3 m_3 \sin(2q_2 + q_3)\big];
\end{aligned}
$$

- $$
\begin{aligned}
C_{13} = \frac{1}{2}\big[&I_{c12}\dot{q}_1 \cos(2q_2 + 2q_3) + I_{c21}\dot{q}_1 \cos(2q_2 \\
&+ 2q_3) + I_{c11}\dot{q}_1 \sin(2q_2 + 2q_3) \\
&- I_{c22}\dot{q}_1 \sin(2q_2 + 2q_3) + I_{c13}\dot{q}_2 \cos(q_2 + q_3) \\
&+ I_{c13}\dot{q}_3 \cos(q_2 + q_3) - I_{c23}\dot{q}_2 \sin(q_2 + q_3) \\
&- I_{c23}\dot{q}_3 \sin(q_2 + q_3) - \dot{q}_1 l_3^2 m_3 \sin(2q_2 \\
&+ 2q_3) - \dot{q}_1 l_2 l_3 m_3 \sin(2q_2 + q_3) \\
&- \dot{q}_1 l_2 l_3 m_3 \sin(q_3)\big];
\end{aligned}
$$

- $$
\begin{aligned}
C_{21} = -\dot{q}_1\Big\{\frac{1}{2}\big[&I_{c12}\cos(2q_2 + 2q_3) + I_{c21}\cos(2q_2 \\
&+ 2q_3) + I_{c11}\sin(2q_2 + 2q_3) - I_{c22}\sin(2q_2 \\
&+ 2q_3) + I_{b12}\cos(2q_2) + I_{b21}\cos(2q_2) \\
&+ I_{b11}\sin(2q_2) - I_{b22}\sin(2q_2) \\
&- l_3^2 m_3 \sin(2q_2 + 2q_3) - l_2^2 m_2 \sin(2q_2) \\
&- l_2^2 m_3 \sin(2q_2) - l_2 l_3 m_3 \sin(2q_2 + q_3)\big]\Big\};
\end{aligned}
$$

- $$
\begin{aligned}
C_{22} = -\dot{q}_1\Big\{\frac{1}{2}\big[&I_{c13}\cos(q_2 + q_3) - I_{c31}\cos(q_2 \\
&+ q_3) - I_{c23}\sin(q_2 + q_3) + I_{c32}\sin(q_2 + q_3) \\
&+ I_{b13}\cos(q_2) - I_{b31}\cos(q_2) \\
&- I_{b23}\sin(q_2) + I_{b32}\sin(q_2)\big]\Big\} \\
&- \dot{q}_3 l_2 l_3 m_3 \sin(q_3);
\end{aligned}
$$

- $$
\begin{aligned}
C_{23} = -\dot{q}_1\Big\{\frac{1}{2}\big[&I_{c13}\cos(q_2 + q_3) - I_{c31}\cos(q_2 + q_3) \\
&- I_{c23}\sin(q_2 + q_3) + I_{c32}\sin(q_2 + q_3)\big]\Big\} \\
&- \dot{q}_2 l_2 l_3 m_3 \sin(q_3) - \dot{q}_3 l_2 l_3 m_3 \sin(q_3);
\end{aligned}
$$

- $$
\begin{aligned}
C_{31} = \dot{q}_1\Big\{\frac{1}{2}\big[&I_{c22}\sin(2q_2 + 2q_3) - I_{c21}\cos(2q_2 \\
&+ 2q_3) - I_{c11}\sin(2q_2 + 2q_3) - I_{c12}\cos(2q_2 \\
&+ 2q_3) + l_3^2 m_3 \sin(2q_2 + 2q_3) \\
&+ l_2 l_3 m_3 \sin(q_3) + l_2 l_3 m_3 \sin(2q_2 + q_3)\big]\Big\};
\end{aligned}
$$

- $$
\begin{aligned}
C_{32} = \dot{q}_2 l_2 l_3 m_3 \sin(q_3) - \dot{q}_1\Big\{\frac{1}{2}\big[&I_{c13}\cos(q_2 + q_3) \\
&- I_{c31}\cos(q_2 + q_3) - I_{c23}\sin(q_2 + q_3) \\
&+ I_{c32}\sin(q_2 + q_3)\big]\Big\};
\end{aligned}
$$

- $$
\begin{aligned}
C_{33} = -\dot{q}_1\Big\{\frac{1}{2}\big[&I_{c13}\cos(q_2 + q_3) - I_{c31}\cos(q_2 + q_3) \\
&- I_{c23}\sin(q_2 + q_3) + I_{c32}\sin(q_2 + q_3)\big]\Big\}.
\end{aligned}
$$

Gravitational vector $\mathbf{g}(\mathbf{q})$.

$$
\mathbf{g}(\mathbf{q}) = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \end{bmatrix},
$$

where

- $g_1 = -c_1 g m_3 (l_2 s_2 + l_3 s_{23}) - c_1 g l_2 m_2 s_2$;
- $g_2 = -g m_3 s_1 (l_2 s_2 + l_3 s_{23}) - g l_2 m_2 s_1 s_2$;
- $g_3 = g m_3 \left(c_2 l_2 + c_{23} l_3\right) + c_2 g l_2 m_2$.

Frictional force $\mathbf{f}(\dot{\mathbf{q}})$.

$$
\mathbf{f}(\dot{\mathbf{q}}) = \mathbf{f}_v \dot{\mathbf{q}} + \mathbf{f}_d(\dot{\mathbf{q}}),
$$

where

- $\mathbf{f}_v = \mathrm{diag}(v_i)$;
- $\mathbf{f}_d(\dot{\mathbf{q}}) = \mathbf{K_d}\ \mathrm{sign}(\dot{\mathbf{q}})$;
- $\mathbf{K}_d = \mathrm{diag}(k_i)$;
- v_i, k_i are the positive constants.

Simulation results

This part presents simulation results for link 2 and link 3 under various reference signals and operating conditions. Figures 8–11 display the control input, tracking performance, and tracking error of both links using a sine wave under ideal and disturbed conditions. Similarly, Figures 12–15 show the corresponding results for a square wave input.

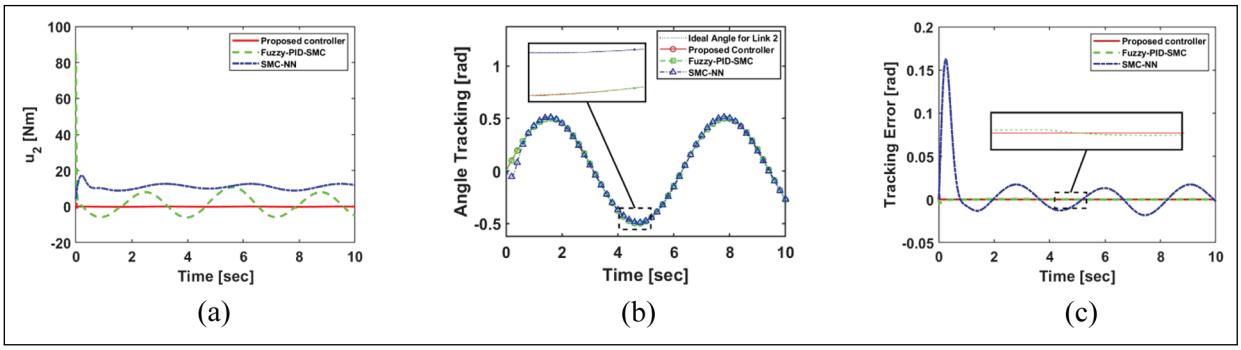


Figure 8. Tracking performance of link 2 with ideal sine wave: (a) control input, (b) angle tracking, and (c) tracking error.

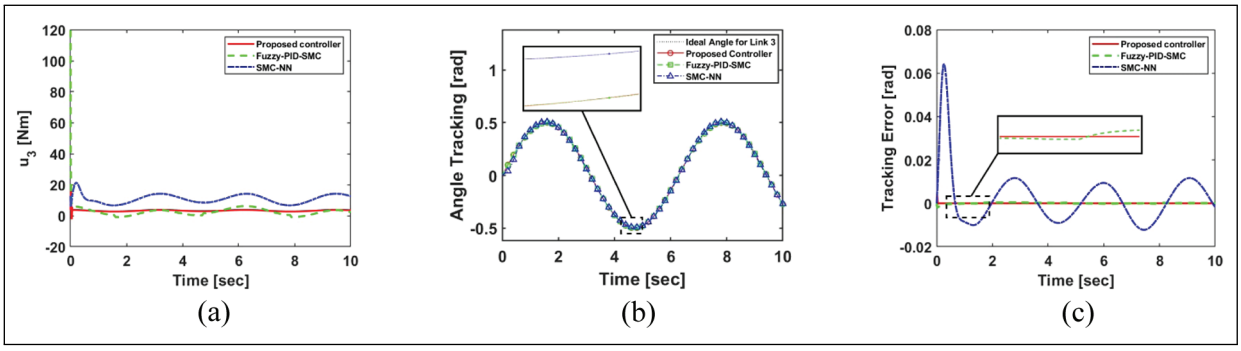


Figure 9. Tracking performance of link 3 with ideal sine wave: (a) control input, (b) angle tracking, and (c) tracking error.

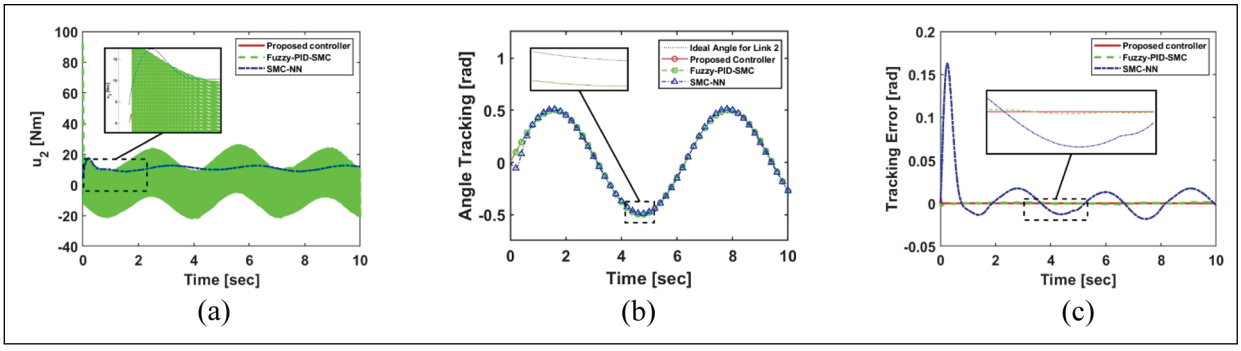


Figure 10. Tracking performance of link 2 with disturbed sine wave: (a) control input, (b) angle tracking, and (c) tracking error.

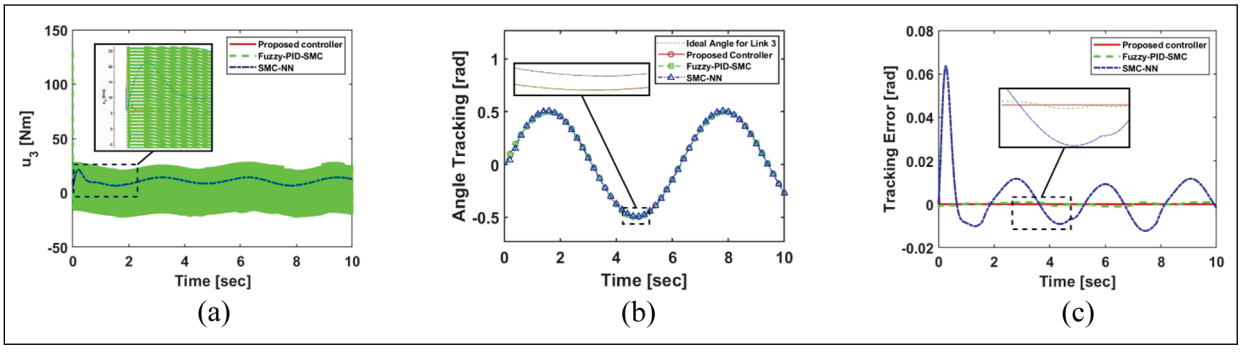


Figure 11. Tracking performance of link 3 with disturbed sine wave: (a) control input, (b) angle tracking, and (c) tracking error.

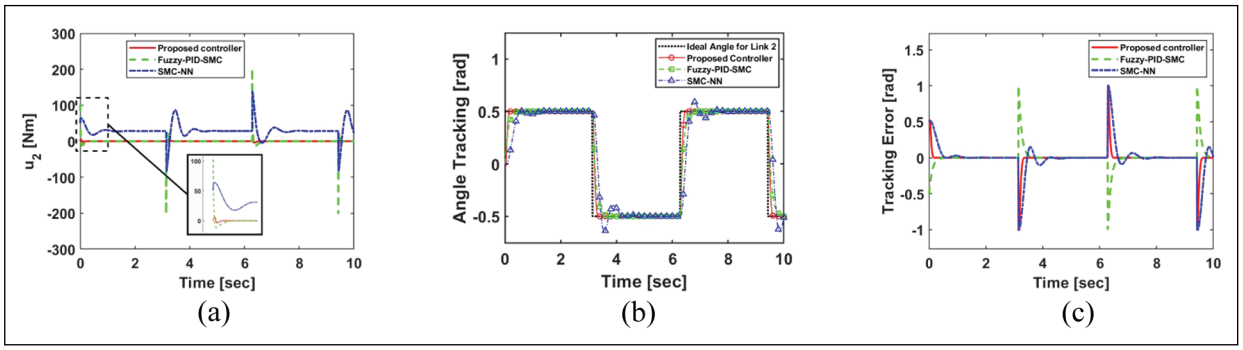


Figure 12. Tracking performance of link 2 with ideal square wave: (a) control input, (b) angle tracking, and (c) tracking error.

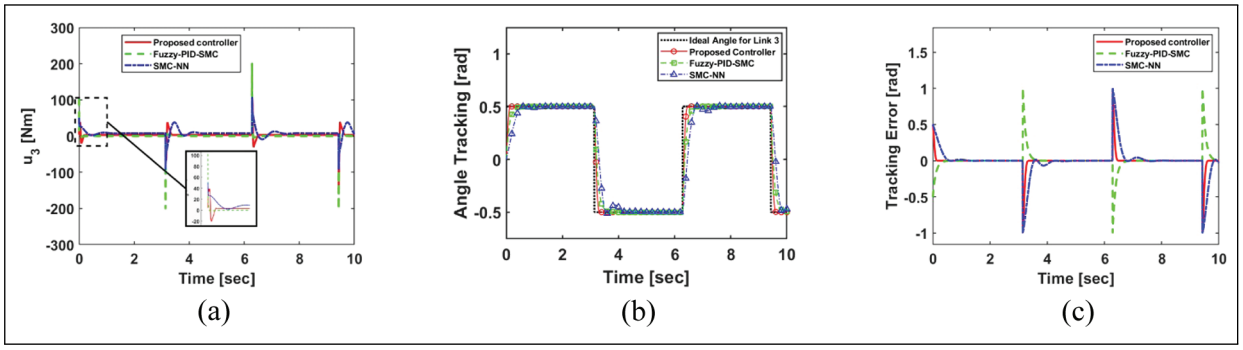


Figure 13. Tracking performance of link 3 with ideal square wave: (a) control input, (b) angle tracking, and (c) tracking error.

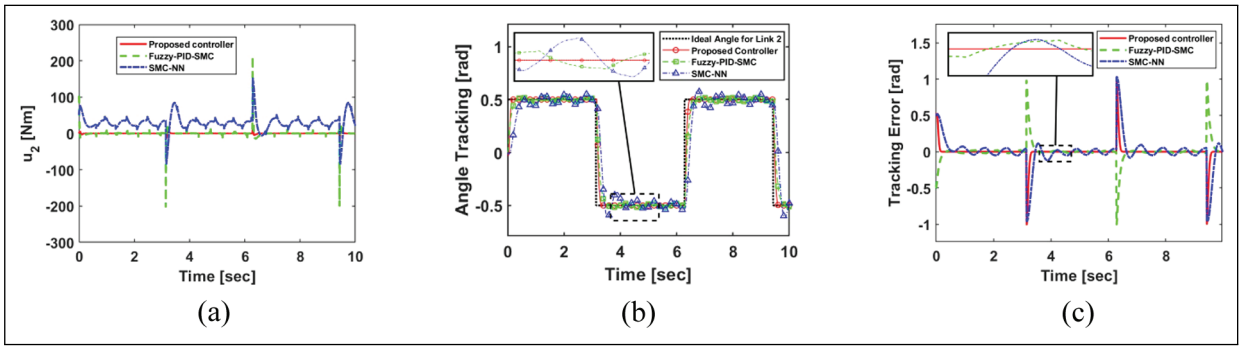


Figure 14. Tracking performance of link 2 with disturbed square wave: (a) control input, (b) angle tracking, and (c) tracking error.

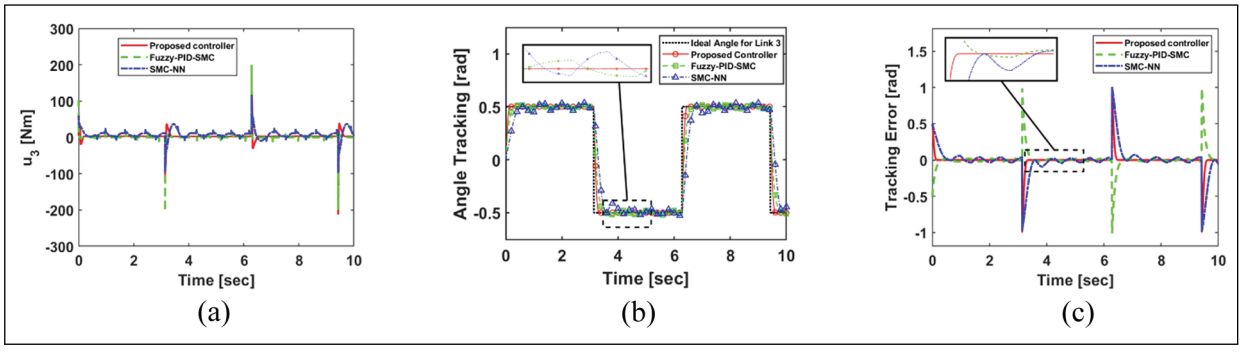


Figure 15. Tracking performance of link 3 with disturbed square wave: (a) control input, (b) angle tracking, and (c) tracking error.